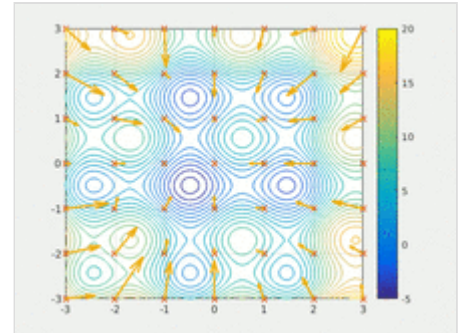




Particle swarm optimization

In computational science, **particle swarm optimization (PSO)**^[1] is a computational method that optimizes a problem by iteratively trying to improve a candidate solution with regard to a given measure of quality. It solves a problem by having a population of candidate solutions, here dubbed particles, and moving these particles around in the search-space according to simple mathematical formulae over the particle's position and velocity. Each particle's movement is influenced by its local best known position, but is also guided toward the best known positions in the search-space, which are updated as better positions are found by other particles. This is expected to move the swarm toward the best solutions.



A particle swarm searching for the global minimum of a function

PSO is originally attributed to Kennedy, Eberhart and Shi^{[2][3]} and was first intended for simulating social behaviour,^[4] as a stylized representation of the movement of organisms in a bird flock or fish school. The algorithm was simplified and it was observed to be performing optimization. The book by Kennedy and Eberhart^[5] describes many philosophical aspects of PSO and swarm intelligence. An extensive survey of PSO applications is made by Poli.^{[6][7]} In 2017, a comprehensive review on theoretical and experimental works on PSO has been published by Bonyadi and Michalewicz.^[1]

PSO is a metaheuristic as it makes few or no assumptions about the problem being optimized and can search very large spaces of candidate solutions. Also, PSO does not use the gradient of the problem being optimized, which means PSO does not require that the optimization problem be differentiable as is required by classic optimization methods such as gradient descent and quasi-newton methods. However, metaheuristics such as PSO do not guarantee an optimal solution is ever found.

Algorithm

A basic variant of the PSO algorithm works by having a population (called a swarm) of candidate solutions (called particles). These particles are moved around in the search-space according to a few simple formulae.^[8] The movements of the particles are guided by their own best-known position in the search-space as well as the entire swarm's best-known position. When improved positions are being discovered these will then come to guide the movements of the swarm. The process is repeated and by doing so it is hoped, but not guaranteed, that a satisfactory solution will eventually be discovered.

Formally, let $f: \mathbb{R}^n \rightarrow \mathbb{R}$ be the cost function which must be minimized. The function takes a candidate solution as an argument in the form of a vector of real numbers and produces a real number as output which indicates the objective function value of the given candidate solution. The gradient of f is not known. The goal is to find a solution $\mathbf{a}$ for which $f(\mathbf{a}) \leq f(\mathbf{b})$ for all $\mathbf{b}$ in the search-space, which would mean $\mathbf{a}$ is the global minimum.

Let S be the number of particles in the swarm, each having a position $\mathbf{x}_i \in \mathbb{R}^n$ in the search-space and a velocity $\mathbf{v}_i \in \mathbb{R}^n$. Let $\mathbf{p}_i$ be the best known position of particle i and let $\mathbf{g}$ be the best known position of the entire swarm. A basic PSO algorithm to minimize the cost function is then:^[9]

```

for each particle  $i = 1, \dots, S$  do
    Initialize the particle's position with a uniformly distributed random vector:  $\mathbf{x}_i \sim U(\mathbf{b}_{lo}, \mathbf{b}_{up})$ 
    Initialize the particle's best known position to its initial position:  $\mathbf{p}_i \leftarrow \mathbf{x}_i$ 
    if  $f(\mathbf{p}_i) < f(\mathbf{g})$  then
        update the swarm's best known position:  $\mathbf{g} \leftarrow \mathbf{p}_i$ 
    Initialize the particle's velocity:  $\mathbf{v}_i \sim U(-|\mathbf{b}_{up}-\mathbf{b}_{lo}|, |\mathbf{b}_{up}-\mathbf{b}_{lo}|)$ 
while a termination criterion is not met do:
    for each particle  $i = 1, \dots, S$  do
        for each dimension  $d = 1, \dots, n$  do
            Pick random numbers:  $r_p, r_g \sim U(0,1)$ 
            Update the particle's velocity:  $\mathbf{v}_{i,d} \leftarrow w \mathbf{v}_{i,d} + \varphi_p r_p (\mathbf{p}_{i,d} - \mathbf{x}_{i,d}) + \varphi_g r_g (\mathbf{g}_d - \mathbf{x}_{i,d})$ 
            Update the particle's position:  $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$ 
            if  $f(\mathbf{x}_i) < f(\mathbf{p}_i)$  then
                Update the particle's best known position:  $\mathbf{p}_i \leftarrow \mathbf{x}_i$ 
            if  $f(\mathbf{p}_i) < f(\mathbf{g})$  then
                Update the swarm's best known position:  $\mathbf{g} \leftarrow \mathbf{p}_i$ 

```

The values $\mathbf{b}_{lo}$ and $\mathbf{b}_{up}$ represent the lower and upper boundaries of the search-space respectively. The w parameter is the inertia weight. The parameters φ_p and φ_g are often called cognitive coefficient and social coefficient.

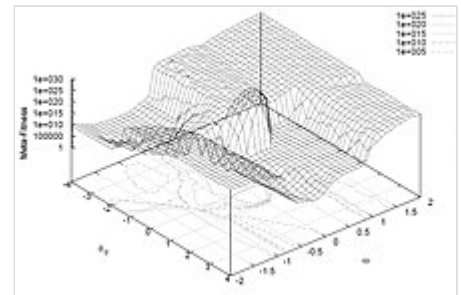
The termination criterion can be the number of iterations performed, or a solution where the adequate objective function value is found.^[10] The parameters w , φ_p , and φ_g are selected by the practitioner and control the behaviour and efficacy of the PSO method (below).

Parameter selection

The choice of PSO parameters can have a large impact on optimization performance. Selecting PSO parameters that yield good performance has therefore been the subject of much research.^{[11][12][13][14][15][16][17][18][19]}

To prevent divergence ("explosion") the inertia weight must be smaller than 1. The two other parameters can be then derived thanks to the constriction approach,^[16] or freely selected, but the analyses suggest convergence domains to constrain them. Typical values are in **[1, 3]**.

The PSO parameters can also be tuned by using another overlaying optimizer, a concept known as meta-optimization,^{[20][21][22][23]} or even fine-tuned during the optimization, e.g., by means of fuzzy logic.^{[24][25]}



Performance landscape showing how a simple PSO variant performs in aggregate on several benchmark problems when varying two PSO parameters.

Parameters have also been tuned for various optimization scenarios.^{[26][27]}

Neighbourhoods and topologies

The topology of the swarm defines the subset of particles with which each particle can exchange information.^[28] The basic version of the algorithm uses the global topology as the swarm communication structure.^[10] This topology allows all particles to communicate with all the other particles, thus the whole swarm share the same best position $\mathbf{g}$ from a single particle. However, this approach might lead the swarm to be trapped into a local minimum,^[29] thus different topologies have been used to control the flow of information among particles. For instance, in local topologies, particles only share information with a subset of particles.^[10] This subset can be a geometrical one^[30] – for example "the m nearest particles" – or, more often, a social one, i.e. a set of particles that is not depending on any distance. In such cases, the PSO variant is said to be local best (vs global best for the basic PSO).

A commonly used swarm topology is the ring, in which each particle has just two neighbours, but there are many others.^[10] The topology is not necessarily static. In fact, since the topology is related to the diversity of communication of the particles,^[31] some efforts have been done to create adaptive topologies (SPSO,^[32] APSO,^[33] stochastic star,^[34] TRIBES,^[35] Cyber Swarm,^[36] and C-PSO^[37])

By using the ring topology, PSO can attain generation-level parallelism, significantly enhancing the evolutionary speed.^[38]

Inner workings

There are several schools of thought as to why and how the PSO algorithm can perform optimization.

A common belief amongst researchers is that the swarm behaviour varies between exploratory behaviour, that is, searching a broader region of the search-space, and exploitative behaviour, that is, a locally oriented search so as to get closer to a (possibly local) optimum. This school of thought has been prevalent since the inception of PSO.^{[3][4][12][16]} This school of thought contends that the PSO algorithm and its parameters must be chosen so as to properly balance between exploration and exploitation to avoid premature convergence to a local optimum yet still ensure a good rate of convergence to the optimum. This belief is the precursor of many PSO variants, see below.

Another school of thought is that the behaviour of a PSO swarm is not well understood in terms of how it affects actual optimization performance, especially for higher-dimensional search-spaces and optimization problems that may be discontinuous, noisy, and time-varying. This school of thought merely tries to find PSO algorithms and parameters that cause good performance regardless of how the swarm behaviour can be interpreted in relation to e.g. exploration and exploitation. Such studies have led to the simplification of the PSO algorithm, see below.

Convergence

In relation to PSO the word *convergence* typically refers to two different definitions:

- Convergence of the sequence of solutions (aka, stability analysis, converging) in which all particles have converged to a point in the search-space, which may or may not be the optimum,
- Convergence to a local optimum where all personal bests **p** or, alternatively, the swarm's best known position **g**, approaches a local optimum of the problem, regardless of how the swarm behaves.

Convergence of the sequence of solutions has been investigated for PSO.^{[15][16][17]} These analyses have resulted in guidelines for selecting PSO parameters that are believed to cause convergence to a point and prevent divergence of the swarm's particles (particles do not move unboundedly and will converge to somewhere). However, the analyses were criticized by Pedersen^[22] for being oversimplified as they assume the swarm has only one particle, that it does not use stochastic variables and that the points of attraction, that is, the particle's best known position **p** and the swarm's best known position **g**, remain constant throughout the optimization process. However, it was shown^[39] that these simplifications do not affect the boundaries found by these studies for parameter where the swarm is convergent. Considerable effort has been made in recent years to weaken the modeling assumption utilized during the stability analysis of PSO,^[40] with the most recent generalized result applying to numerous PSO variants and utilized what was shown to be the minimal necessary modeling assumptions.^[41]

Convergence to a local optimum has been analyzed for PSO in^[42] and^[43] It has been proven that PSO needs some modification to guarantee finding a local optimum.

This means that determining the convergence capabilities of different PSO algorithms and parameters still depends on empirical results. One attempt at addressing this issue is the development of an "orthogonal learning" strategy for an improved use of the information already existing in the relationship between **p** and **g**, so as to form a leading converging exemplar and to be effective with any PSO topology. The aims are to improve the performance of PSO overall, including faster global convergence, higher solution quality, and stronger robustness.^[44] However, such studies do not provide theoretical evidence to actually prove their claims.

Adaptive mechanisms

Without the need for a trade-off between convergence ('exploitation') and divergence ('exploration'), an adaptive mechanism can be introduced. Adaptive particle swarm optimization (APSO)^[45] features better search efficiency than standard PSO. APSO can perform global search over the entire search space with a higher convergence speed. It enables automatic control of the inertia weight, acceleration coefficients, and other algorithmic parameters at the run time, thereby improving the search effectiveness and efficiency at the same time. Also, APSO can act on the globally best particle to jump out of the likely local optima. However, APSO will introduce new algorithm parameters, it does not introduce additional design or implementation complexity nonetheless.

Besides, through the utilization of a scale-adaptive fitness evaluation mechanism, PSO can efficiently address computationally expensive optimization problems.^[46]

Variants

Numerous variants of even a basic PSO algorithm are possible. For example, there are different ways to initialize the particles and velocities (e.g. start with zero velocities instead), how to dampen the velocity, only update $\mathbf{p}_i$ and $\mathbf{g}$ after the entire swarm has been updated, etc. Some of these choices and their possible performance impact have been discussed in the literature.^[14]

A series of standard implementations have been created by leading researchers, "intended for use both as a baseline for performance testing of improvements to the technique, as well as to represent PSO to the wider optimization community. Having a well-known, strictly-defined standard algorithm provides a valuable point of comparison which can be used throughout the field of research to better test new advances."^[10] The latest is Standard PSO 2011 (SPSO-2011).^[47]

Hybridization

New and more sophisticated PSO variants are also continually being introduced in an attempt to improve optimization performance. There are certain trends in that research; one is to make a hybrid optimization method using PSO combined with other optimizers,^{[48][49][50]} e.g., combined PSO with biogeography-based optimization,^[51] and the incorporation of an effective learning method.^[44]

Alleviate premature convergence

Another research trend is to try to alleviate premature convergence (that is, optimization stagnation), e.g. by reversing or perturbing the movement of the PSO particles,^{[19][52][53][54]} another approach to deal with premature convergence is the use of multiple swarms^[55] (multi-swarm optimization). The multi-swarm approach can also be used to implement multi-objective optimization.^[56] Finally, there are developments in adapting the behavioural parameters of PSO during optimization.^{[45][24]}

Simplifications

Another school of thought is that PSO should be simplified as much as possible without impairing its performance; a general concept often referred to as Occam's razor. Simplifying PSO was originally suggested by Kennedy^[4] and has been studied more extensively,^{[18][21][22][57]} where it appeared that optimization performance was improved, and the parameters were easier to tune and they performed more consistently across different optimization problems.

Another argument in favour of simplifying PSO is that metaheuristics can only have their efficacy demonstrated empirically by doing computational experiments on a finite number of optimization problems. This means a metaheuristic such as PSO cannot be proven correct and this increases the risk of making errors in its description and implementation. A good example of this^[58] presented a promising variant of a genetic algorithm (another popular metaheuristic) but it was later found to be defective as it was strongly biased in its optimization search towards similar values for different dimensions in the search space, which happened to be the optimum of the benchmark problems considered. This bias was because of a programming error, and has now been fixed.^[59]

Bare Bones PSO

Initialization of velocities may require extra inputs. The Bare Bones PSO variant^[60] has been proposed in 2003 by James Kennedy, and does not need to use velocity at all.

In this variant of PSO one dispenses with the velocity of the particles and instead updates the positions of the particles using the following simple rule,

$$\vec{x}_i = G\left(\frac{\vec{p}_i + \vec{g}}{2}, \|\vec{p}_i - \vec{g}\|\right),$$

where $\vec{x}_i$, $\vec{p}_i$ are the position and the best position of the particle i ; $\vec{g}$ is the global best position; $G(\vec{x}, \sigma)$ is the normal distribution with the mean $\vec{x}$ and standard deviation σ ; and where $\|\dots\|$ signifies the norm of a vector.

Accelerated Particle Swarm Optimization

Another simpler variant is the accelerated particle swarm optimization (APSO),^[61] which also does not need to use velocity and can speed up the convergence in many applications. A simple demo code of APSO is available.^[62]

In this variant of PSO one dispenses with both the particle's velocity and the particle's best position. The particle position is updated according to the following rule,

$$\vec{x}_i \leftarrow (1 - \beta)\vec{x}_i + \beta\vec{g} + \alpha L\vec{u},$$

where $\vec{u}$ is a random uniformly distributed vector, L is the typical length of the problem at hand, and $\beta \sim 0.1 - 0.7$ and $\alpha \sim 0.1 - 0.5$ are the parameters of the method. As a refinement of the method one can decrease α with each iteration, $\alpha_n = \alpha_0 \gamma^n$, where n is the number of the iteration and $0 < \gamma < 1$ is the decrease control parameter.

Multi-objective optimization

PSO has also been applied to multi-objective problems,^{[63][64][65]} in which the objective function comparison takes Pareto dominance into account when moving the PSO particles and non-dominated solutions are stored so as to approximate the pareto front.

Binary, discrete, and combinatorial

As the PSO equations given above work on real numbers, a commonly used method to solve discrete problems is to map the discrete search space to a continuous domain, to apply a classical PSO, and then to demap the result. Such a mapping can be very simple (for example by just using rounded values) or more sophisticated.^[66]

However, it can be noted that the equations of movement make use of operators that perform four actions:

- computing the difference of two positions. The result is a velocity (more precisely a displacement)
- multiplying a velocity by a numerical coefficient
- adding two velocities
- applying a velocity to a position

Usually a position and a velocity are represented by n real numbers, and these operators are simply $-$, $*$, $+$, and again $+$. But all these mathematical objects can be defined in a completely different way, in order to cope with binary problems (or more generally discrete ones), or even combinatorial ones.^{[67][68][69][70]} One approach is to redefine the operators based on sets.^[71]

See also

- [Artificial bee colony algorithm](#)
- [Bees algorithm](#)
- [Derivative-free optimization](#)
- [Multi-swarm optimization](#)
- [Particle filter](#)
- [Swarm intelligence](#)
- [Fish School Search](#)
- [Dispersive flies optimisation](#)

References

1. Bonyadi, M. R.; Michalewicz, Z. (2017). "Particle swarm optimization for single objective continuous space problems: a review". *Evolutionary Computation*. **25** (1): 1–54. doi:10.1162/EVCO_r_00180 (https://doi.org/10.1162%2FEVCO_r_00180). PMID 26953883 (<https://pubmed.ncbi.nlm.nih.gov/26953883>). S2CID 8783143 (<https://api.semanticscholar.org/CorpusID:8783143>).
2. Kennedy, J.; Eberhart, R. (1995). "Particle Swarm Optimization". *Proceedings of IEEE International Conference on Neural Networks*. Vol. IV. pp. 1942–1948. doi:10.1109/ICNN.1995.488968 (<https://doi.org/10.1109%2FICNN.1995.488968>).
3. Shi, Y.; Eberhart, R.C. (1998). "A modified particle swarm optimizer". *Proceedings of IEEE International Conference on Evolutionary Computation*. pp. 69–73. doi:10.1109/ICEC.1998.699146 (<https://doi.org/10.1109%2FICEC.1998.699146>).
4. Kennedy, J. (1997). "The particle swarm: social adaptation of knowledge". *Proceedings of IEEE International Conference on Evolutionary Computation*. pp. 303–308. doi:10.1109/ICEC.1997.592326 (<https://doi.org/10.1109%2FICEC.1997.592326>).
5. Kennedy, J.; Eberhart, R.C. (2001). *Swarm Intelligence*. Morgan Kaufmann. ISBN 978-1-55860-595-4.
6. Poli, R. (2007). "An analysis of publications on particle swarm optimisation applications" (<https://web.archive.org/web/20110716231935/http://cswww.essex.ac.uk/technical-reports/2007/tr-csm469.pdf>) (PDF). *Technical Report CSM-469*. Archived from the original (<http://cswww.essex.ac.uk/technical-reports/2007/tr-csm469.pdf>) (PDF) on 2011-07-16. Retrieved 2010-05-03.
7. Poli, R. (2008). "Analysis of the publications on the applications of particle swarm optimisation" (<http://downloads.hindawi.com/archive/2008/685175.pdf>) (PDF). *Journal of Artificial Evolution and Applications*. **2008**: 1–10. doi:10.1155/2008/685175 (<https://doi.org/10.1155%2F2008%2F685175>).
8. Zhang, Y. (2015). "A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications" (<http://www.hindawi.com/journals/mpe/2015/931256>). *Mathematical Problems in Engineering*. **2015**: 931256.
9. Clerc, M. (2012). "Standard Particle Swarm Optimisation" (http://hal.archives-ouvertes.fr/docs/00/76/49/96/PDF/SPSO_descriptions.pdf) (PDF). *HAL Open Access Archive*.

10. Bratton, Daniel; Kennedy, James (2007). "Defining a Standard for Particle Swarm Optimization". *2007 IEEE Swarm Intelligence Symposium* (<http://www.cil.pku.edu.cn/resources/psopaper/src/2007SPSO.pdf>) (PDF). pp. 120–127. doi:10.1109/SIS.2007.368035 (<http://doi.org/10.1109%2FSIS.2007.368035>). ISBN 978-1-4244-0708-8. S2CID 6217309 (<http://api.semanticscholar.org/CorpusID:6217309>).
11. Taherkhani, M.; Safabakhsh, R. (2016). "A novel stability-based adaptive inertia weight for particle swarm optimization". *Applied Soft Computing*. **38**: 281–295. doi:10.1016/j.asoc.2015.10.004 (<https://doi.org/10.1016%2Fj.asoc.2015.10.004>).
12. Shi, Y.; Eberhart, R.C. (1998). "Parameter selection in particle swarm optimization". *Proceedings of Evolutionary Programming VII (EP98)*. pp. 591–600.
13. Eberhart, R.C.; Shi, Y. (2000). "Comparing inertia weights and constriction factors in particle swarm optimization" (<https://www.researchgate.net/publication/3865142>). *Proceedings of the Congress on Evolutionary Computation*. Vol. 1. pp. 84–88.
14. Carlisle, A.; Dozier, G. (2001). "An Off-The-Shelf PSO" (https://web.archive.org/web/20030503203304/http://antho.huntingdon.edu/publications/Off-The-Shelf_PSO.pdf) (PDF). *Proceedings of the Particle Swarm Optimization Workshop*. pp. 1–6. Archived from the original (http://antho.huntingdon.edu/publications/Off-The-Shelf_PSO.pdf) (PDF) on 2003-05-03.
15. van den Bergh, F. (2001). *An Analysis of Particle Swarm Optimizers* (<http://bee22.com/resources/Bergh%202006.pdf>) (PDF) (PhD thesis). University of Pretoria, Faculty of Natural and Agricultural Science.
16. Clerc, M.; Kennedy, J. (2002). "The particle swarm - explosion, stability, and convergence in a multidimensional complex space". *IEEE Transactions on Evolutionary Computation*. **6** (1): 58–73. CiteSeerX 10.1.1.460.6608 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.460.6608>). doi:10.1109/4235.985692 (<https://doi.org/10.1109%2F4235.985692>).
17. Trelea, I.C. (2003). "The Particle Swarm Optimization Algorithm: convergence analysis and parameter selection". *Information Processing Letters*. **85** (6): 317–325. doi:10.1016/S0020-0190(02)00447-7 (<https://doi.org/10.1016%2FS0020-0190%2802%2900447-7>).
18. Bratton, D.; Blackwell, T. (2008). "A Simplified Recombinant PSO" (<http://downloads.hindawi.com/archive/2008/654184.pdf>) (PDF). *Journal of Artificial Evolution and Applications*. **2008**: 1–10. doi:10.1155/2008/654184 (<https://doi.org/10.1155%2F2008%2F654184>).
19. Evers, G. (2009). *An Automatic Regrouping Mechanism to Deal with Stagnation in Particle Swarm Optimization* (<https://web.archive.org/web/20110518164430/http://www.georgeevers.org/publications.htm>). The University of Texas - Pan American, Department of Electrical Engineering. Archived from the original (<http://www.georgeevers.org/publications.htm>) (Master's thesis) on 2011-05-18. Retrieved 2010-05-05.
20. Meissner, M.; Schmuker, M.; Schneider, G. (2006). "Optimized Particle Swarm Optimization (OPSO) and its application to artificial neural network training" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1464136>). *BMC Bioinformatics*. **7** (1): 125. doi:10.1186/1471-2105-7-125 (<https://doi.org/10.1186%2F1471-2105-7-125>). PMC 1464136 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1464136>). PMID 16529661 (<https://pubmed.ncbi.nlm.nih.gov/16529661>).
21. Pedersen, M.E.H. (2010). *Tuning & Simplifying Heuristical Optimization* (<https://web.archive.org/web/20200213130101/https://pdfs.semanticscholar.org/a5d2/8c26a2e2824170d67b69f14120cf67cabe26.pdf>) (PDF). University of Southampton, School of Engineering Sciences, Computational Engineering and Design Group. S2CID 107805461 (<https://api.semanticscholar.org/CorpusID:107805461>). Archived from the original (<https://pdfs.semanticscholar.org/a5d2/8c26a2e2824170d67b69f14120cf67cabe26.pdf>) (PhD thesis) on 2020-02-13.
22. Pedersen, M.E.H.; Chipperfield, A.J. (2010). "Simplifying particle swarm optimization". *Applied Soft Computing*. **10** (2): 618–628. CiteSeerX 10.1.1.149.8300 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.149.8300>). doi:10.1016/j.asoc.2009.08.029 (<https://doi.org/10.1016%2Fj.asoc.2009.08.029>).

23. Mason, Karl; Duggan, Jim; Howley, Enda (2018). "A Meta Optimisation Analysis of Particle Swarm Optimisation Velocity Update Equations for Watershed Management Learning". *Applied Soft Computing*. **62**: 148–161. doi:10.1016/j.asoc.2017.10.018 (<https://doi.org/10.1016%2Fj.asoc.2017.10.018>).
24. Nobile, M.S; Cazzaniga, P.; Besozzi, D.; Colombo, R.; Mauri, G.; Pasi, G. (2018). "Fuzzy Self-Tuning PSO: a settings-free algorithm for global optimization". *Swarm and Evolutionary Computation*. **39**: 70–85. doi:10.1016/j.swevo.2017.09.001 (<https://doi.org/10.1016%2Fj.swevo.2017.09.001>). hdl:10446/106467 (<https://hdl.handle.net/10446%2F106467>).
25. Nobile, M.S; Pasi, G.; Cazzaniga, P.; Besozzi, D.; Colombo, R.; Mauri, G. (2015). "Proactive particles in swarm optimization: a self-tuning algorithm based on fuzzy logic". *Proceedings of the 2015 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE 2015), Istanbul (Turkey)*. pp. 1–8. doi:10.1109/FUZZ-IEEE.2015.7337957 (<https://doi.org/10.1109%2FFUZZ-IEEE.2015.7337957>).
26. Cazzaniga, P.; Nobile, M.S.; Besozzi, D. (2015). "The impact of particles initialization in PSO: parameter estimation as a case in point, (Canada)". *Proceedings of IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology*. doi:10.1109/CIBCB.2015.7300288 (<https://doi.org/10.1109%2FCIBCB.2015.7300288>).
27. Pedersen, M.E.H. (2010). "Good parameters for particle swarm optimization". *Technical Report HL1001*. CiteSeerX 10.1.1.298.4359 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.298.4359>).
28. Kennedy, J.; Mendes, R. (2002). "Population structure and particle swarm performance". *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No.02TH8600)*. Vol. 2. pp. 1671–1676 vol.2. CiteSeerX 10.1.1.114.7988 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.114.7988>). doi:10.1109/CEC.2002.1004493 (<https://doi.org/10.1109%2FCEC.2002.1004493>). ISBN 978-0-7803-7282-5. S2CID 14364974 (<https://api.semanticscholar.org/CorpusID:14364974>).
29. Mendes, R. (2004). *Population Topologies and Their Influence in Particle Swarm Performance* (<https://pdfs.semanticscholar.org/d224/80b09d1f0759fb20e0fb0bd2de205457c8bc.pdf>) (PhD thesis). Universidade do Minho.
30. Suganthan, Ponnuthurai N. "Particle swarm optimiser with neighbourhood operator (<https://ieeexplore.ieee.org/abstract/document/785514/>).". *Evolutionary Computation, 1999. CEC 99. Proceedings of the 1999 Congress on*. Vol. 3. IEEE, 1999.
31. Oliveira, M.; Pinheiro, D.; Andrade, B.; Bastos-Filho, C.; Menezes, R. (2016). "Communication Diversity in Particle Swarm Optimizers". *Swarm Intelligence. Lecture Notes in Computer Science*. Vol. 9882. pp. 77–88. doi:10.1007/978-3-319-44427-7_7 (https://doi.org/10.1007%2F978-3-319-44427-7_7). ISBN 978-3-319-44426-0. S2CID 37588745 (<https://api.semanticscholar.org/CorpusID:37588745>).
32. SPSO Particle Swarm Central (<http://www.particleswarm.info>)
33. Almasi, O. N. and Khooban, M. H. (2017). A parsimonious SVM model selection criterion for classification of real-world data sets via an adaptive population-based algorithm. *Neural Computing and Applications*, 1-9. <https://doi.org/10.1007/s00521-017-2930-y> (<https://link.springer.com/article/10.1007/s00521-017-2930-y>)
34. Miranda, V., Keko, H. and Duque, Á. J. (2008). Stochastic Star Communication Topology in Evolutionary Particle Swarms (EPSO) (<https://repositorio.inesctec.pt/bitstream/123456789/1561/1/PS-05818.pdf>). *International Journal of Computational Intelligence Research (IJCIR)*, Volume 4, Number 2, pp. 105-116
35. Clerc, M. (2006). *Particle Swarm Optimization*. ISTE (International Scientific and Technical Encyclopedia), 2006
36. Yin, P., Glover, F., Laguna, M., & Zhu, J. (2011). A Complementary Cyber Swarm Algorithm (http://leeds-faculty.colorado.edu/glover/fred%20pubs/428%20-%20A_complementary_cyber_swarm_algorithm_pub%20version%20w%20open%20et%20al.pdf). *International Journal of Swarm Intelligence Research (IJSIR)*, 2(2), 22-41

37. Elshamy, W.; Rashad, H.; Bahgat, A. (2007). "Clubs-based Particle Swarm Optimization" (http://web.archive.org/web/20131023025232/http://people.cis.ksu.edu/~welshamy/pubs/ieee_sis07.pdf) (PDF). *IEEE Swarm Intelligence Symposium 2007 (SIS2007)*. Honolulu, HI. pp. 289–296. Archived from the original (http://people.cis.ksu.edu/~welshamy/pubs/ieee_sis07.pdf) (PDF) on 2013-10-23. Retrieved 2012-04-27.
38. Jian-Yu, Li (2021). "Generation-Level Parallelism for Evolutionary Computation: A Pipeline-Based Parallel Particle Swarm Optimization" (<https://ieeexplore.ieee.org/document/9248594>). *IEEE Transactions on Cybernetics*. **51** (10): 4848-4859. doi:10.1109/TCYB.2020.3028070 (<https://doi.org/10.1109/2FTCYB.2020.3028070>).
39. Cleghorn, Christopher W (2014). "Particle Swarm Convergence: Standardized Analysis and Topological Influence". *Swarm Intelligence*. Lecture Notes in Computer Science. Vol. 8667. pp. 134–145. doi:10.1007/978-3-319-09952-1_12 (https://doi.org/10.1007/2F978-3-319-09952-1_12). ISBN 978-3-319-09951-4.
40. Liu, Q (2015). "Order-2 stability analysis of particle swarm optimization". *Evolutionary Computation*. **23** (2): 187–216. doi:10.1162/EVCO_a_00129 (https://doi.org/10.1162/2FEVCO_a_00129). PMID 24738856 (<https://pubmed.ncbi.nlm.nih.gov/24738856>). S2CID 25471827 (<https://api.semanticscholar.org/CorpusID:25471827>).
41. Cleghorn, Christopher W.; Engelbrecht, Andries. (2018). "Particle Swarm Stability: A Theoretical Extension using the Non-Stagnate Distribution Assumption". *Swarm Intelligence*. **12** (1): 1–22. doi:10.1007/s11721-017-0141-x (<https://doi.org/10.1007/2Fs11721-017-0141-x>). hdl:2263/62934 (<https://hdl.handle.net/2263/2F62934>). S2CID 9778346 (<https://api.semanticscholar.org/CorpusID:9778346>).
42. Van den Bergh, F. "A convergence proof for the particle swarm optimizer" ([https://repository.up.ac.za/bitstream/handle/2263/17262/VanDenBergh_Convergence\(2010\).pdf?sequence=1](https://repository.up.ac.za/bitstream/handle/2263/17262/VanDenBergh_Convergence(2010).pdf?sequence=1)) (PDF). *Fundamenta Informaticae*.
43. Bonyadi, Mohammad reza.; Michalewicz, Z. (2014). "A locally convergent rotationally invariant particle swarm optimization algorithm" (<https://espace.library.uq.edu.au/view/UQ:396054/ERAUQ396054.pdf>) (PDF). *Swarm Intelligence*. **8** (3): 159–198. doi:10.1007/s11721-014-0095-1 (<https://doi.org/10.1007/2Fs11721-014-0095-1>). S2CID 2261683 (<https://api.semanticscholar.org/CorpusID:2261683>).
44. Zhan, Z-H.; Zhang, J.; Li, Y; Shi, Y-H. (2011). "Orthogonal Learning Particle Swarm Optimization" (<http://eprints.gla.ac.uk/44801/1/44801.pdf>) (PDF). *IEEE Transactions on Evolutionary Computation*. **15** (6): 832–847. doi:10.1109/TEVC.2010.2052054 (<https://doi.org/10.1109/2FTEVC.2010.2052054>).
45. Zhan, Z-H.; Zhang, J.; Li, Y; Chung, H.S-H. (2009). "Adaptive Particle Swarm Optimization" (<http://eprints.gla.ac.uk/7645/1/7645.pdf>) (PDF). *IEEE Transactions on Systems, Man, and Cybernetics*. **39** (6): 1362–1381. doi:10.1109/TSMCB.2009.2015956 (<https://doi.org/10.1109/2FTSMCB.2009.2015956>). PMID 19362911 (<https://pubmed.ncbi.nlm.nih.gov/19362911>). S2CID 11191625 (<https://api.semanticscholar.org/CorpusID:11191625>).
46. Wang, Ye-Qun; Li, Jian-Yu; Chen, Chun-Hua; Zhang, Jun; Zhan, Zhi-Hui (September 2023). "Scale adaptive fitness evaluation-based particle swarm optimisation for hyperparameter and architecture optimisation in neural networks and deep learning" (<https://doi.org/10.1049/cit2.12106>). *CAAI Transactions on Intelligence Technology*. **8** (3): 849-862. doi:10.1049/cit2.12106 (<https://doi.org/10.1049/2Fcit2.12106>).
47. Zambrano-Bigiarini, M.; Clerc, M.; Rojas, R. (2013). "Standard Particle Swarm Optimisation 2011 at CEC-2013: A baseline for future PSO improvements". *2013 IEEE Congress on Evolutionary Computation*. Evolutionary Computation (CEC), 2013 IEEE Congress on. pp. 2337–2344. doi:10.1109/CEC.2013.6557848 (<https://doi.org/10.1109/2FCEC.2013.6557848>). ISBN 978-1-4799-0454-9. S2CID 206553432 (<https://api.semanticscholar.org/CorpusID:206553432>).

48. Lovbjerg, M.; Krink, T. (2002). "The LifeCycle Model: combining particle swarm optimisation, genetic algorithms and hillclimbers" (http://www.lovbjerghome.dk/Morten/EvaLife/TK_PPSN_2002_LifeCycle_PSO_HC_GA.pdf) (PDF). *Proceedings of Parallel Problem Solving from Nature VII (PPSN)*. pp. 621–630.
49. Niknam, T.; Amiri, B. (2010). "An efficient hybrid approach based on PSO, ACO and k-means for cluster analysis". *Applied Soft Computing*. **10** (1): 183–197. doi:10.1016/j.asoc.2009.07.001 (<https://doi.org/10.1016%2Fj.asoc.2009.07.001>).
50. Zhang, Wen-Jun; Xie, Xiao-Feng (2003). DEPSO: hybrid particle swarm with differential evolution operator (<http://www.wiomax.com/team/xie/paper/SMCC03.pdf>). *IEEE International Conference on Systems, Man, and Cybernetics (SMCC)*, Washington, DC, USA: 3816-3821.
51. Zhang, Y.; Wang, S. (2015). "Pathological Brain Detection in Magnetic Resonance Imaging Scanning by Wavelet Entropy and Hybridization of Biogeography-based Optimization and Particle Swarm Optimization" (<https://doi.org/10.2528%2Fpier15040602>). *Progress in Electromagnetics Research*. **152**: 41–58. doi:10.2528/pier15040602 (<https://doi.org/10.2528%2Fpier15040602>).
52. Lovbjerg, M.; Krink, T. (2002). "Extending Particle Swarm Optimisers with Self-Organized Criticality" (http://www.lovbjerghome.dk/Morten/EvaLife/ML_CEC2002_SOCPSO.pdf) (PDF). *Proceedings of the Fourth Congress on Evolutionary Computation (CEC)*. Vol. 2. pp. 1588–1593.
53. Xinchao, Z. (2010). "A perturbed particle swarm algorithm for numerical optimization". *Applied Soft Computing*. **10** (1): 119–124. doi:10.1016/j.asoc.2009.06.010 (<https://doi.org/10.1016%2Fj.asoc.2009.06.010>).
54. Xie, Xiao-Feng; Zhang, Wen-Jun; Yang, Zhi-Lian (2002). A dissipative particle swarm optimization (<http://www.wiomax.com/team/xie/paper/CEC02.pdf>). *Congress on Evolutionary Computation (CEC)*, Honolulu, HI, USA: 1456-1461.
55. Cheung, N. J.; Ding, X.-M.; Shen, H.-B. (2013). "OptiFel: A Convergent Heterogeneous Particle Sarm Optimization Algorithm for Takagi-Sugeno Fuzzy Modeling". *IEEE Transactions on Fuzzy Systems*. **22** (4): 919–933. doi:10.1109/TFUZZ.2013.2278972 (<https://doi.org/10.1109%2FTFUZZ.2013.2278972>). S2CID 27974467 (<https://api.semanticscholar.org/CorpusID:27974467>).
56. Nobile, M.; Besozzi, D.; Cazzaniga, P.; Mauri, G.; Pescini, D. (2012). "A GPU-Based Multi-Swarm PSO Method for Parameter Estimation in Stochastic Biological Systems Exploiting Discrete-Time Target Series". *Evolutionary Computation, Machine Learning and Data Mining in Bioinformatics. Lecture Notes in Computer Science*. Vol. 7264. pp. 74–85. doi:10.1007/978-3-642-29066-4_7 (https://doi.org/10.1007%2F978-3-642-29066-4_7).
57. Yang, X.S. (2008). *Nature-Inspired Metaheuristic Algorithms*. Luniver Press. ISBN 978-1-905986-10-1.
58. Tu, Z.; Lu, Y. (2004). "A robust stochastic genetic algorithm (StGA) for global numerical optimization". *IEEE Transactions on Evolutionary Computation*. **8** (5): 456–470. doi:10.1109/TEVC.2004.831258 (<https://doi.org/10.1109%2FTEVC.2004.831258>). S2CID 22382958 (<https://api.semanticscholar.org/CorpusID:22382958>).
59. Tu, Z.; Lu, Y. (2008). "Corrections to "A Robust Stochastic Genetic Algorithm (StGA) for Global Numerical Optimization". *IEEE Transactions on Evolutionary Computation*. **12** (6): 781. doi:10.1109/TEVC.2008.926734 (<https://doi.org/10.1109%2FTEVC.2008.926734>). S2CID 2864886 (<https://api.semanticscholar.org/CorpusID:2864886>).
60. Kennedy, James (2003). "Bare bones particle swarms". *Proceedings of the 2003 IEEE Swarm Intelligence Symposium. SIS'03 (Cat. No.03EX706)*. pp. 80–87. doi:10.1109/SIS.2003.1202251 (<https://doi.org/10.1109%2FSIS.2003.1202251>). ISBN 0-7803-7914-4. S2CID 37185749 (<https://api.semanticscholar.org/CorpusID:37185749>).

61. X. S. Yang, S. Deb and S. Fong, Accelerated particle swarm optimization and support vector machine for business optimization and applications (<https://arxiv.org/abs/1203.6577>), NDT 2011, Springer CCIS 136, pp. 53-66 (2011).
62. "Search Results: APSO - File Exchange - MATLAB Central" (<http://www.mathworks.com/matlabcentral/fileexchange/?term=APSO>).
63. Parsopoulos, K.; Vrahatis, M. (2002). "Particle swarm optimization method in multiobjective problems". *Proceedings of the ACM Symposium on Applied Computing (SAC)*. pp. 603–607. doi:10.1145/508791.508907 (<https://doi.org/10.1145%2F508791.508907>).
64. Coello Coello, C.; Salazar Lechuga, M. (2002). "MOPSO: A Proposal for Multiple Objective Particle Swarm Optimization" (<http://portal.acm.org/citation.cfm?id=1252327>). *Congress on Evolutionary Computation (CEC'2002)*. pp. 1051–1056.
65. Mason, Karl; Duggan, Jim; Howley, Enda (2017). "Multi-objective dynamic economic emission dispatch using particle swarm optimisation variants". *Neurocomputing*. **270**: 188–197. doi:10.1016/j.neucom.2017.03.086 (<https://doi.org/10.1016%2Fj.neucom.2017.03.086>).
66. Roy, R., Dehuri, S., & Cho, S. B. (2012). A Novel Particle Swarm Optimization Algorithm for Multi-Objective Combinatorial Optimization Problem (<http://sclab.yonsei.ac.kr/publications/Papers/IJ/A%20Novel%20Particle%20Swarm%20Optimization%20Algorithm%20for%20Multi-Objective%20Combinatorial%20Optimization%20Problem.pdf>). 'International Journal of Applied Metaheuristic Computing (IJAMC)', 2(4), 41-57
67. Kennedy, J. & Eberhart, R. C. (1997). A discrete binary version of the particle swarm algorithm (http://ahmetcevahircinar.com.tr/wp-content/uploads/2017/02/A_discrete_binary_version_of_the_particle_swarm_algorithm.pdf), Conference on Systems, Man, and Cybernetics, Piscataway, NJ: IEEE Service Center, pp. 4104-4109
68. Clerc, M. (2004). Discrete Particle Swarm Optimization, illustrated by the Traveling Salesman Problem (https://link.springer.com/chapter/10.1007/978-3-540-39930-8_8), New Optimization Techniques in Engineering, Springer, pp. 219-239
69. Clerc, M. (2005). Binary Particle Swarm Optimisers: toolbox, derivations, and mathematical insights, Open Archive HAL (<http://hal.archives-ouvertes.fr/hal-00122809/en/>)
70. Jarboui, B.; Damak, N.; Siarry, P.; Rebai, A. (2008). "A combinatorial particle swarm optimization for solving multi-mode resource-constrained project scheduling problems". *Applied Mathematics and Computation*. **195**: 299–308. doi:10.1016/j.amc.2007.04.096 (<https://doi.org/10.1016%2Fj.amc.2007.04.096>).
71. Chen, Wei-neng; Zhang, Jun (2010). "A novel set-based particle swarm optimization method for discrete optimization problem". *IEEE Transactions on Evolutionary Computation*. **14** (2): 278–300. CiteSeerX 10.1.1.224.5378 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.224.5378>). doi:10.1109/tevc.2009.2030331 (<https://doi.org/10.1109%2Ftevc.2009.2030331>). S2CID 17984726 (<https://api.semanticscholar.org/CorpusID:17984726>).

External links

- Particle Swarm Central (<http://www.particleswarm.info>) is a repository for information on PSO. Several source codes are freely available.
- A brief video (<https://vimeo.com/17407010>) of particle swarms optimizing three benchmark functions.
- Simulation of PSO convergence in a two-dimensional space (Matlab). (<http://www.mathworks.com/matlabcentral/fileexchange/11559-particle-swarm-optimization-simulation>)
- Applications (<http://www.vocal.com/particle-swarm-optimization/>) of PSO.
- Liu, Yang (2009). "Automatic calibration of a rainfall–runoff model using a fast and elitist multi-objective particle swarm algorithm". *Expert Systems with Applications*. **36** (5): 9533–9538. doi:10.1016/j.eswa.2008.10.086 (<https://doi.org/10.1016%2Fj.eswa.2008.10.086>).

- [Links to PSO source code \(http://www.adaptivebox.net/research/bookmark/psocodes_link.html\)](http://www.adaptivebox.net/research/bookmark/psocodes_link.html)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Particle_swarm_optimization&oldid=1232289477"

■